Chapter 4

Artificial Neural Networks (ANN)

4.1 Introduction

Neural networks represent a brain metaphor for information processing. This model is biologically inspired rather than an exact replica of how the brain actually functions. Neural networks have been shown to be very promising systems in many forecasting applications and business classification applications due to their ability to "learn" from the data, their nonparametric nature (i.e., no rigid assumptions), and their ability to generalize. *Neural computing* refers to a pattern recognition methodology for machine learning. The resulting model from neural computing is often called an *Artificial Neural Network (ANN)* or a *neural network*. Neural networks have been used in many business applications for pattern recognition, forecasting, prediction, and classification. Neural network computing is a key component of any data mining tool kit. Applications of neural networks abound in finance, marketing, manufacturing, operations, information systems, and so on. Therefore, we devote this chapter to developing a better understanding of neural network models, methods, and applications.

The human brain possesses bewildering capabilities for information processing and problem solving that modern computer cannot compete with in many aspects. It has been postulated that a model or a system that is enlightened and supported by the results from brain research, with a structure similar to that of biological neural networks, could exhibit similar intelligent functionality. Based on this bottom-up postulation, ANN (also known as connectionist models, parallel distributed processing models, neuromorphic systems, or simply neural networks) have been developed as biologically inspired and plausible models for various tasks [9].

Biological neural networks are composed of many massively interconnected primitive biological neurons. Each neuron possesses *axons* and *dendrites*, finger-like projections that enable the neuron to communicate with its neighboring *neurons* by transmitting and receiving

electrical and chemical signals. More or less resembling the structure of their counterparts, ANN is composed of interconnected, simple processing elements called *artificial neurons*. In processing information, the processing elements in an ANN operate concurrently and collectively in a similar fashion to biological neurons [5]. ANN possesses some desirable traits similar to those of biological neural networks, such as the capabilities of learning, self-organization, and fault tolerance. ANN have been investigated by researchers for more than half a century. The formal study of ANN began with the pioneering work of McCulloch and Pitts in 1943. Stimulated by results of biological experiments and observations, McCulloch and Pitts (1943) introduced a simple model of a binary artificial neuron that captures some functions of a living neuron [8]. Considering information processing machines as a means for modeling the brain, McCulloch and Pitts built their neural networks model using a large number of interconnected binary artificial neurons. Led by a school of researchers, neural network research was quite popular in the late 1950s and early 1960s. After a thorough analysis of an early neural network model (called the **perceptron**, which used no hidden layer) as well as a pessimistic evaluation of the research potential by Minsky and Papert in 1969, the interest in neural networks diminished [7]. During the past two decades, there has been an exciting resurgence in the studies of ANN due to the introduction of new network topologies, new activation functions, and new learning algorithms, as well as progress in neuroscience and cognitive science. On the one hand, advances in theory and methodology have overcome many obstacles that hindered neural network research a few decades ago. Evidenced by the appealing results of numerous studies, neural networks are gaining acceptance and popularity. On the other hand, as complex problems solvers, ANN has been applied to solve numerous problems in a variety of application settings. The desirable features in neural information processing make neural networks attractive for solving complex problems. The initial success in neural network applications has inspired renewed interest from industry and business.

4.1.1 Biological Neurons

Neurons (*special cells)* are the core components of the nervous system. In biological terms neuron is an electrically excitable cell that processes and transmits information by electrical and

chemical signaling. There are different types of neurons like, Sensory neurons which respond to touch, sound, light and numerous other stimuli affecting cells of the sensory organs that then send signals to the spinal cord and brain. Motor neurons receive signals from the brain and spinal cord and cause muscle contractions and affect glands functions. Interneuron connects neurons to other neurons within the same region of the brain or spinal cord. Neurons are partitioned into groups called networks. Each network contains several thousand highly interconnected neurons. Thus, the brain can be viewed as a collection of neural networks. The ability to learn and react to changes in our environment requires intelligence. The brain and the central nervous system control thinking and intelligent behavior. This is the reason why people who suffer brain damage have difficulty learning and reacting to changing environments. Even so, undamaged parts of the brain can often compensate with new learning.

NOTE

It is claimed that the human central nervous system is comprised of about 1.3×10^{10} neurons. The estimated number of neurons in a human brain is 50 to 150 billion, of which there are more than 100 different kinds.

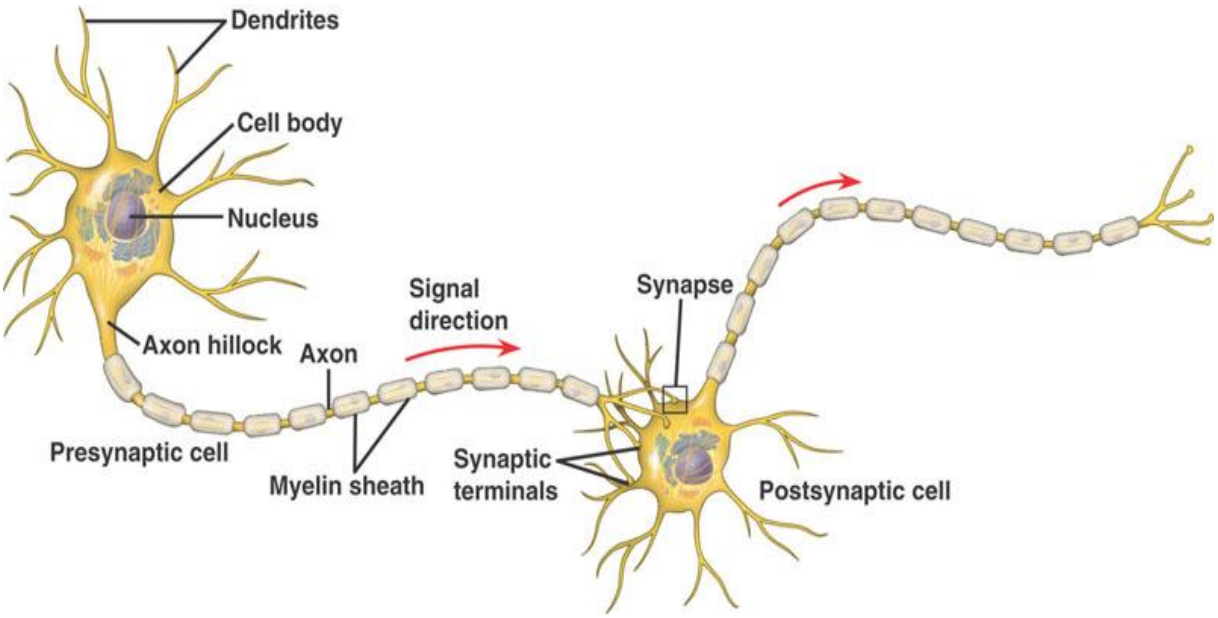


Fig 4.1: Structure of Biological Neuron

As represented in the figure 4.1, a neuron has a roughly spherical cell body called ***Soma***, a group of filaments that arise from the cell body called ***Dendrite***, and an extension of cell body called ***Axon***. Axon is one that carries a neurons output to the dendrites of other neurons. But the amount of information transfer that takes place between two neurons is solely dependent on a material called ***Synapses*** which interfaces these two neurons. The Synapses forms the key element for Neural information Processing. A neuron collects signals at its synapses by summing all the excitatory and inhibitory influences acting on it. If the excitatory influences are dominant, then the neuron fires and sends this message to other neurons via the outgoing synapses [1][5].

4.2 Artificial Neural Networks

As mentioned earlier, an Artificial Neural Network (ANN) is an information processing paradigm that was first designed by *McCulloch and Pitts in 1943,* inspired by the way biological nervous systems, such as the brain works. As mentioned above the ANNs are usually used for pattern recognition or data classification. But how do they learn the pattern or trends in the data. Of course, the ANNs, like humans have to learn by examples. Once they have learned it, the discovery of patterns or predictions can be done very effectively. An ANN is configured for a specific application, such as pattern recognition or data classification, through a learning process. In the context of data mining, we are interested in the use of neural networks for machine learning and information processing.

4.3 Elements of ANN

An ANN is composed of processing elements (artificial neurons) organized in different layers to form the network's structure. The basic processing unit of an ANN is the neuron. A number of neurons are organized into a network. There are different topologies which exist depending on the way these neurons are organized in the network e.g. Back propagation. There are basically three layers for a typical ANN. These are shown in figure 4.2. They include

1) Input Layer
2) Hidden/Intermediate/Middle layer

3) Output Layer

NOTE

The most fascinating feature of an ANN is the parallel processing involved in it working. When information is processed many of the processing elements perform their computations at the same time. This parallel processing resembles the way the brain works.

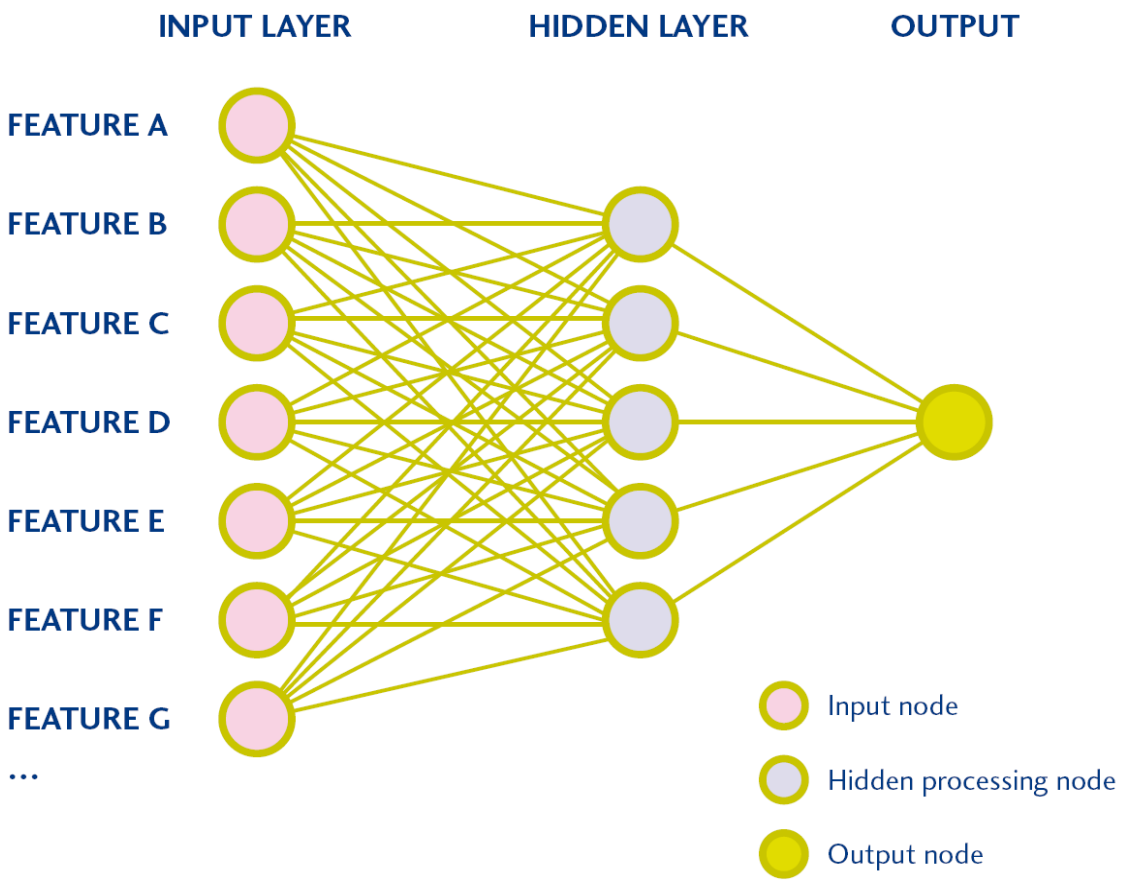


Fig 4.2: Basic elements of ANN

4.3.1 Input Layer

The input layer is where the artificial neuron receives inputs. The artificial neuron receives inputs analogous to the electrochemical impulses the dendrites of biological neurons receive from other neurons. Each input corresponds to a single attribute and there may be many inputs (features) which go into the input layer. Several types of data, such as text, pictures, and voice,

can be used as inputs. Preprocessing may be needed to convert the data to meaningful inputs from symbolic data or to scale the data E.g. Consider a software program implanted using ANN logic which predicts whether a customer can be provided a loan or not. For this the software will have to have some inputs such as income level, age, ownership etc. These data are preprocessed for converting them to meaningful inputs (i.e. inputs in the form of numerical values) and this is passed into the neural network architecture as feature inputs. This can be seen from figure 4.2.

4.3.2 Hidden/Intermediate/Middle Layer

The hidden layer is a layer that interfaces the Input layer with the output layer. Typically it would be fine to say that all data processing and decision making takes places in this layer. The hidden layer converts the original input in the problem into some higher level combinations of such inputs. All connections between neurons have an associated weight. *The 'Connection Weights' form the backbone of input data processing.* They show the relative importance of each input to the processing element, and ultimately the outputs. In essence, the "knowledge" possessed by the network is encapsulated in these interconnection weights. The middle layer of ANN mainly uses two functions - Summation function and Transformation Function.

Complex practical applications require one or more hidden layers between the input and output neurons and a correspondingly large number of weights. Many commercial ANN include three and sometimes up to five layers, with each containing 10 to 1,000 processing elements. Some experimental ANN use millions of processing elements. Because each layer increases the training effort exponentially and also increases the computation required, the use of more than three hidden layers is rare in most commercial systems [10].

4.3.3 Summation Function

This function computes the weighted sum of all the input elements entering each processing element. The summation function multiplies each input values by a weight factor and finally sums up all of them and gives the weighted sum as the output (Y). (see Figure 4.3)

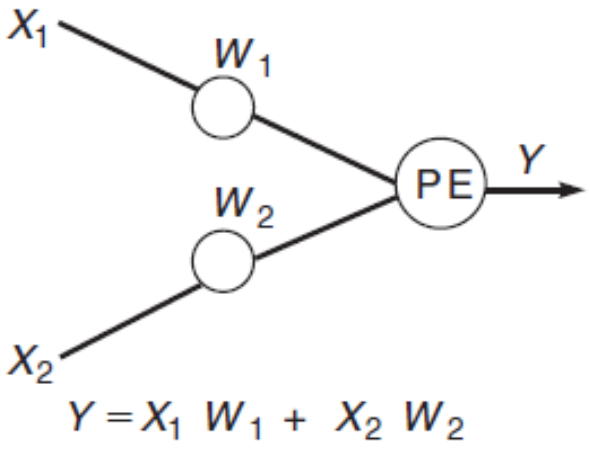


Fig 4.3: Single neuron model

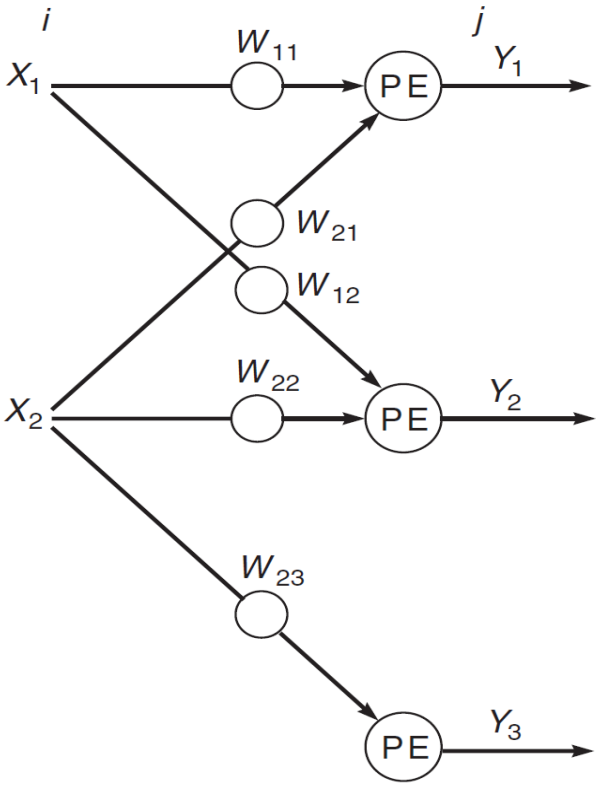


Fig 4.4: Multi-neuron model

Mathematically summation function shown in figure 4.3 can be represented as,

$$Y = \sum_{i=1}^{n} X_i W_i \tag{6.1}$$

Here n represents the number of inputs, $\mathbf{X_i}$ is the $\mathbf{i^{th}}$ input, W_i is the Connection weight for $\mathbf{i^{th}}$ input and Y is the weighted sum.

For the jth neuron of several processing neurons in a layer (see Figure 4.4), the formula becomes,

$$Y_j = \sum_{i=1}^{n} X_i W_{ij} \tag{6.2}$$

Figure 4.4 shows the figurative representation for the equation 6.2.

4.3.4 Activation/Transformation function

The activation function are employed to controls the response of a neural network layer in order to achieve the required functionality. The inputs to any layer in neural network are numerical values. These multiplied by weights to get the response a layer in neural network. Now the job of the activation function is to tune the network output to expected state depending on the summation output. The figure 4.5, gives an idea about the actual location of the activation function module.

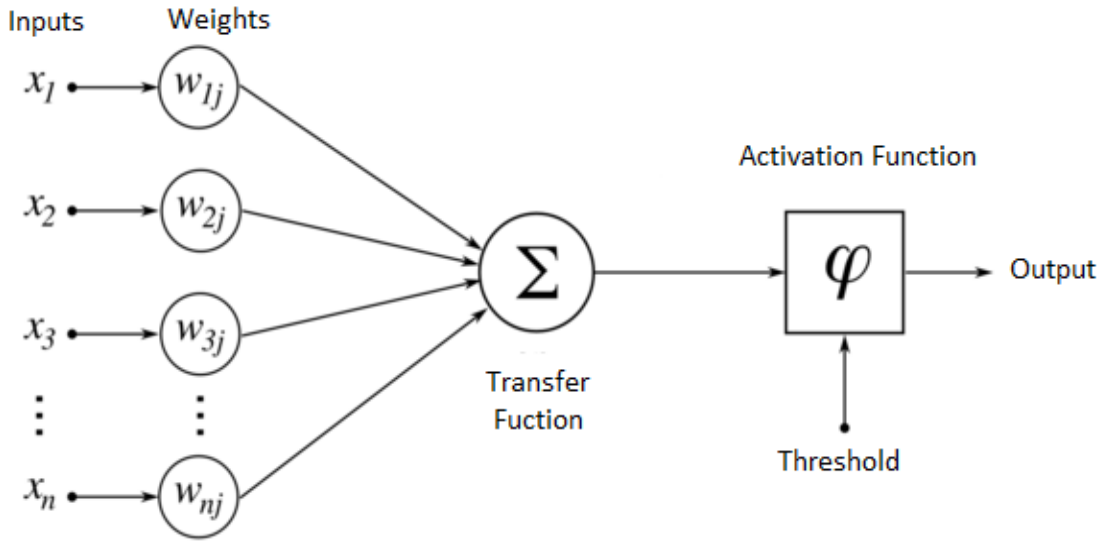


Fig 4.5: Generalized Diagram of Neural Network with Activation function

The activation functions are of different types. Different types of activation functions are chosen based on the distribution of target values. In case of simple perceptrons the ouput can have values 0 or 1(output has only two levels) and so it is logical to select an activation function which provides only two states, whatever be the value at the output node. Hence, as discussed previously, the simple perceptron uses simple thresholding function. In case of multilayer perceprtons the capability to produce a non linear classifier comes from the use of a non linear activation function such as a sigmoid function [5]. This nonlinearlity is one which makes multilayer perceptrons so powerful. For backpropagation learning, the activation function must be differentiable, and it helps if the function is bounded; the sigmoidal functions such as logistic and tanh and the Gaussian function are the most common choices in that case. Functions such as tanh or arctan that produce both positive and negative values tend to yield faster training than functions that produce only positive values such as logistic, because of better numerical conditioning. For getting the normalized hidden layer output it is always better to go for sigmoid function than threshold activation functions. This is because with sigmoid units, a small change in the weights will usually produce a change in the outputs, which makes it possible to tell whether that change in the weights is good or bad but with threshold activation functions, a small change in the weights will often produce no change in the outputs which is not expected.

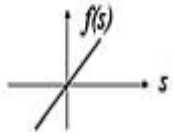
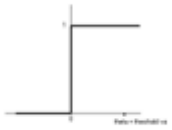
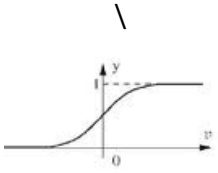
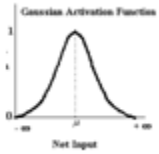
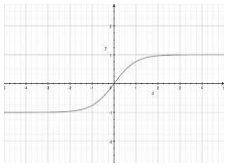
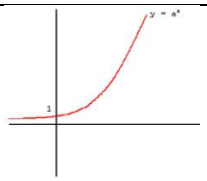
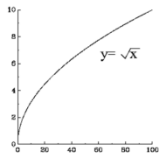
FUNCTION	DEFINITION	RANGE	PLOT
Linear	$f(x) = x$	$[-\infty, +\infty]$	
Step/Threshold	$f(x) = \begin{cases} 1 & x>0 \\ 0 & Otherwise \end{cases}$	[0 ,+1]	
Logistic/sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	[0, +1]	
Gaussian	$f(x) = e^{\left[-\Pi(i_j-\mu_j)^2\right]}$	$[0, +\infty]$	
Hyperbolic tangent	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	[-1, +1]	
Exponential	$f(x) = e^{-x}$	$[0, +\infty]$	
Square root	$f(x) = \sqrt{x}$	$[0, +\infty]$	

Table 4.1: Types of activation functions

It is always better to choose a non linear activation function unless the situation demands a linear activation function. If the output variable has to limited to a value between 0 and 1 including 0 and 1 then it is better to choose logistic function at the output. For categorical outputs requirements it is better to choose softmax activation function. For continuous-valued targets with a bounded range, the logistic and tanh functions can be used, provided you either scale the outputs to the range. If the target values need to be positive but have no known upper bound, you can use an exponential output activation function.

Factors determining the selection of activation function

1) **Derivability** – The function should be derivable or differentiable. This propery should be satisfied because the gradient descent algorithm on the function to find the minimum value of error function can only be applied if the function is differentiable.
2) **Continuity** – A continuous function would have values at all point and this would help in easily finding out the derivative.
3) **Complexity** - The function requires to be nonlinear for higher order problems/complex problems to be feasible for an multi layer perceptron.
4) **Monotonous** – A monotonous increasing function would make sure that the derivative are never zero or negative and hence gradient descent algorithm is made useful.
5) **Boundedness** – This guarantees that the activation output and derivative are finite. This makes interpreting the results for multi layer perceptron much convenient.
6) **Polarity** – The polarity corresponds to the functions sign. Some signals are symmetrical whereas some are bipolar.

If we take the case of sigmoid function (Fig 4.1); it is one among the popular Transformation Function. It is an S shaped transfer function in the range 0 and 1.

$$Y_T = \frac{1}{1+e^{-Y}} \tag{6.3}$$

Where, Y is the output of Summation Function, Y^T is the normalized output. The functions says that, as the value of Y is very large e^{-y} will be very large and the output will approach 1 and as

the e^{-y} will be very small the denominator of the sigmoid function will become very large as a output approach 0.

4.3.5 Output Layer

The output layer is one which provides a solution to the problem. Often, postprocessing of the outputs is required because some networks use two outputs: one for yes and another for no. It is common to have to round the outputs to the nearest 0 or 1. E.g. In case of Loan prediction software mentioned above, the output may be 'Yes' or 'No' and all the output layer will provide is a value 0 or 1 which corresponds to 'Yes' or 'No'. Here it is always better if we could round off these values to 0 or 1. Thus in this case, the output will always be in the interval 0 to 1 irrespective of whether the output of Transformation function is very large or very small. Keep in mind that, the output of the artificial neuron corresponds to signals sent out from a biological neuron over its axon.

NOTE

Sometimes instead of Transformation Function a Threshold value is used. Threshold value is the hurdle value for the output of neuron to trigger the next level of neurons.

4.3.6 Mathematical Viewpoint of ANN

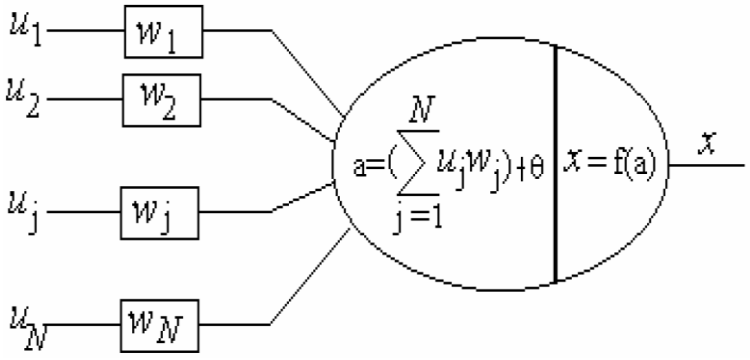


Fig 4.6: Mathematical representation - Artificial Neuron

The artificial neuron given in this figure 4.6 has *N* input, denoted as u_1, u_2 ...u_N. Each line connecting these inputs to the neuron is assigned a weight, which is denoted as w_1, w_2...w_N

respectively. Weights in the artificial model correspond to the synaptic connections in biological neurons. The **Threshold** in artificial neuron is usually represented by θ and the activation corresponding to the graded potential is given by the formula 4.4:

$$a = \left(\sum_{j=1}^{N} w_j u_j \right) + \theta \tag{6.4}$$

The inputs and the weights are real values. A negative value for a weight indicates an inhibitory connection while a positive value indicates an excitatory one. Although in biological neurons, θ has a negative value; it may be assigned a positive value in artificial neuron models. If θ is positive, it is usually referred as bias. For its mathematical convenience we will use (+) sign in the activation formula. Sometimes, the threshold is combined for simplicity into the summation part by assuming an imaginary input u0 =+1 and a connection weight w0 = θ. Hence the activation formula becomes 4.5,

$$a = \sum_{j=0}^{N} w_j u_j \tag{6.5}$$

The output value of the neuron is a function of its activation in an analogy to the firing frequency of the biological neurons:

$$x = f(a) \tag{6.6}$$

4.4 A Short Comparison

It is a fact that, the whole ANN is just the mathematical implementation of the processes in the biological neural Network. First of all let's try to relate the structure of both. Similar to electrochemical impulses the dendrites of biological neurons receive from other neurons, the artificial neuron receives inputs. The output of the artificial neuron corresponds to signals sent out from a biological neuron over its axon.

Biological neural network implement all functionalities by working in a group. I.e. there are many neurons layers between the point where input is applied and the point where the

output is produced. Similarly there are many layers in the ANNs also – Input Layer, Middle Layer (there can be multiple middle layers) and an Output Layer.

Learning in biological systems involves adjustments to the synaptic connections that exist between the neurons. The transmission of a signal from one neuron to another through synapses is a complex chemical process in which specific transmitter substances are released from the sending side of the junction. The effect is to raise or lower the electrical potential inside the body of the receiving cell. If this graded potential reaches a threshold, the neuron fires. Similarly learning in ANNs is achieved by changing the weights (multiplication factor) in the middle layers. Remember the fact told about biological neurons, that it fires if the combined signal strength exceeds a certain threshold. In case of ANNs such functionality is achieved by an Activation function or Threshold Function.

BIOLOGICAL	ARTIFICIAL
Soma	Node
Dendrites	Input
Axon	Output
Synapse	Weight
Slow	Fast
Many neurons(10^9)	Few neurons (a dozen to a hundred of thousands)

Table 4.2: Comparison between biological and artificial neurons

We borrow concepts from the biological world to improve the design of computers. ANN technology is used for complex information processing and machine intelligence. On the other hand, neural networks can also be used as simple biological models to test hypotheses about "real" biological neuronal information processing. Of course, in the context of data mining, we are interested in the use of neural networks for machine learning and information processing.

4.5 Artificial Neural Network Learning

An important consideration in an ANN is the use of an appropriate learning algorithm (or training algorithm). Learning algorithms specify the process by which a neural network learns the underlying relationship between input and outputs, or just among the inputs. [The learning capability of an artificial neuron is achieved by adjusting the weights in accordance to the chosen learning algorithm]. The ANN learns through one of the following ways

- Supervised learning
- Unsupervised learning
- Reinforcement learning

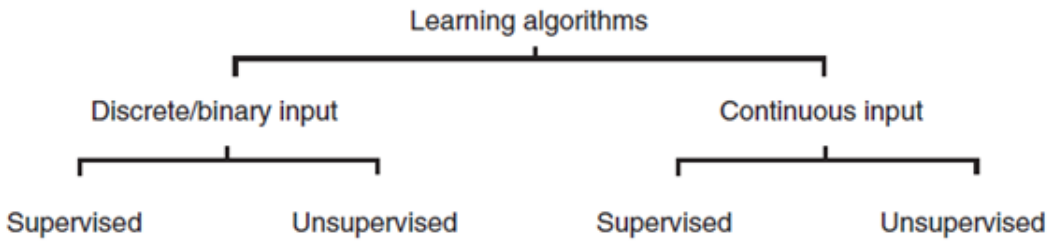


Fig 4.7: Generalized categories of Learning Algorithms

4.5.1 Supervised learning

The term supervised originates from the fact that the desired signals on individual output nodes are provided by an external teacher [2]. Here the algorithm learns from a set of data which is provided to it and which was actually obtained by past experiences. Initially the weights will be randomly selected and for each input the output of the ANN is compared with the expected result, and the weights are modified suitability depending upon the error in the output obtained. As more and more inputs are applied the weights will be changed correspondingly and thereby the output will become more and more accurate (i.e. Actual result will be close to expected result). Thus this type of learning is done when the requirement is to predict information based on previous values. The best-known examples of this technique occur is the backpropagation algorithm, the delta rule, and the perceptron rule.

NOTE

The back propagation learning algorithm is the standard way of implementing supervised training of feed forward neural networks.

4.5.1.1 Supervised Learning Process

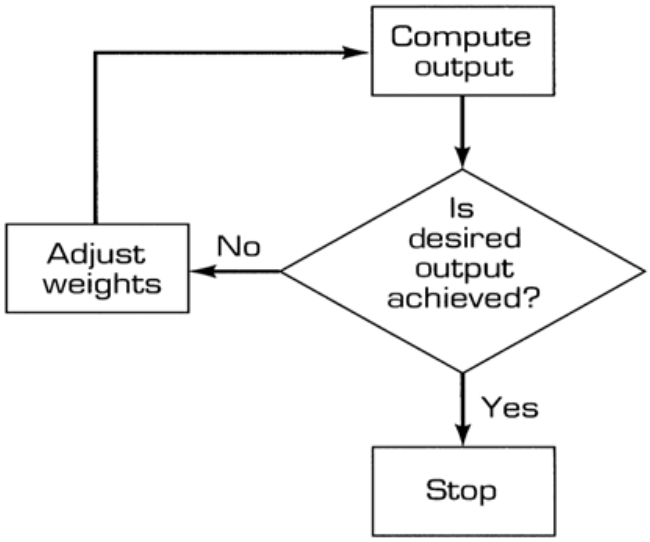


Fig: 4.8: Learning Process of an ANN

In supervised learning, the learning process is inductive; that is, connection weights are derived from existing cases. The usual process of learning involves three tasks are shown in the figure 4.8

1. Compute temporary outputs.
2. Compare outputs with desired targets.
3. Adjust the weights and repeat the process.

When existing outputs are available for comparison, the learning process starts by setting the connection weights, either via rules or randomly. The difference between the actual output (*Y* or *YT*) and the desired output (*Z*) for a given set of inputs is an error called *delta* (in calculus, the Greek symbol delta, Δ, means “difference”). The objective is to minimize the delta (i.e., reduce it to 0 if possible), which is done by adjusting the network’s weights. The key is to change the weights in the right direction; making changes that reduce the delta (i.e., error).We will show

how this is done later. Information processing with an ANN consists of an attempt to recognize patterns of activities (i.e., **pattern recognition**). During the learning stages, the interconnection weights change in response to training data presented to the system. Different ANN compute delta in different ways, depending on the learning algorithm being used. There are hundreds of learning algorithms for various situations and configurations, some of which are discussed later in this chapter.

4.5.2 Unsupervised Learning

Unsupervised Learning is used to look for patterns, groupings or other ways to characterize the data that may lead to understanding of the way the data interrelates. The unsupervised learning network organizes itself internally so that each hidden processing element responds strategically to a different set of input stimuli (or groups of stimuli). Here unlike the supervised learning, there is no a priori set of categories into which the patterns are to be classified; rather, the system must develop its own representation of the input stimuli i.e. No knowledge is supplied about which classifications (i.e., outputs) are correct and which are not. However, by setting model parameters, we can control the number of categories into which a network classifies the inputs. Regardless, a human must examine the final categories to assign meaning and determine the usefulness of the results. Adaptive resonance theory (ART), Kohonen self-organizing feature maps, principle components analysis and statistical measures are examples of unsupervised learning. The working of Kohonen self-organizing feature maps is explained in later chapers.

4.5.3 Reinforcement Learning

Reinforcement learning is less famous compared to the other two. Reinforcement learning is learning what to do – how to map situations to actions – so as to maximize a numerical reward signal. The learner is not told which actions to take, as in most forms of machine learning, but instead must discover which actions yield the most reward by trying them. In the most interesting and challenging cases, actions may affect not only the immediate reward, but also

the next situation and, through that, all subsequent rewards. These two characteristics, trial-and error search and delayed reward are the two most important distinguishing features of reinforcement learning [2]. Game playing, Robot in a maze, Multiple agents, partial observability are application areas where such learning is utilized.

4.5.4 Hebbian Learning

Introduced by Donald Hebb in 1949 - the Hebb's postulate; states that learning was based on the modification of synaptic connections between neurons. Specifically, when an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased. The principles underlying this statement have become known as Hebbian Learning. Virtually, most of the neural network learning techniques can be considered as a variant of the Hebbian learning rule. The basic idea is that if two neurons are active simultaneously, their interconnection must be strengthened [2].

The other types of learning like Perceptron learning and Backpropogation learning will be explained along with the ANN Architecture section.

4.6 Artificial Neural Network Architectures

The operation of the entire neural network model is driven by the task it is designed to address. For instance, neural network models have been used as classifiers, as forecasting tools, and as general optimizers. The basic architecture of ANN consists of three types of neuron layers: input, hidden, and output layers. Neural network classifiers are typically multilayer models in which information is passed from one layer to the next, with the ultimate goal of mapping an input to the network to a specific category, as identified by an output of the network. On the other hand, it can be a single layer of neurons, highly interconnected, and can compute neuron values iteratively until the model converges to a stable state. This stable state would then represent an optimal solution to the problem under analysis. There are several effective neural

network models and algorithms [12]. Some of the most common are *feed-forward networks, backpropagation, associative memory*, and the *recurrent network*

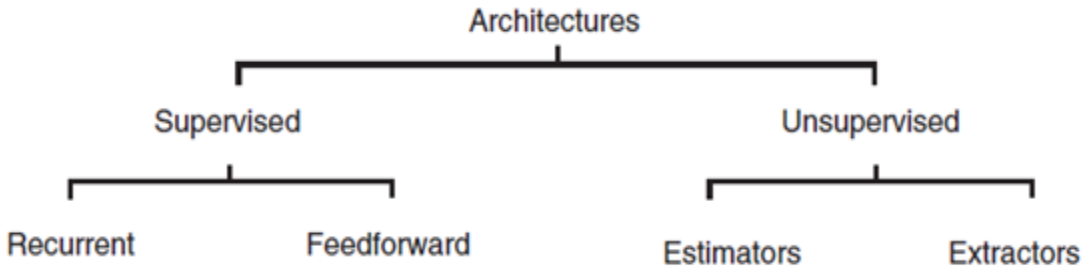


Fig 4.9: Neural Network Architectures

4.6.1 Feedforward Architecture

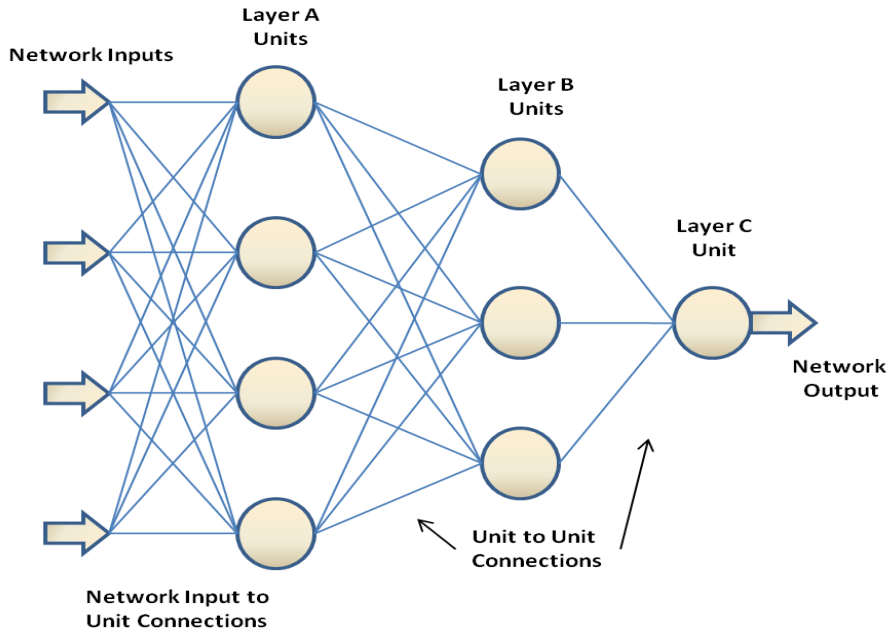


Fig 4.10: Feed Forward Network

The feedforward neural network is one in which information moves only in one direction. Here the data processing can extend over multiple (layers of) units, but no feedback connections are present [4]. Multilayer, feedforward neural networks are a class of models that show promise in classification and forecasting problems. As the name implies, these models structurally consist of multiple layers of neurons. Information is passed through the network in one direction, from

the input layers of the network, through one or more hidden layers, toward the output layer of neurons [12]. Neurons of each layer are connected only to the neurons of the subsequent layer. Figure 4.10 shows a typical feedforward network

4.6.2 Perceptron & Feedforward – Back propagation

A perceptron is the simplest and most widely used algorithms in use. It almost mocks the functionality of a neuron in human brain. A simple perceptron has a number of external inputs given to it and one or more outputs depending on whether it is a single class perceptron or multiclass perceptron. Usually, the input values are Boolean (just two possible values 1 and 0, true and false), but they can also be any number. The output of the perceptron, however, is always Boolean. Figure 4.11 shows simple perceptron architecture.

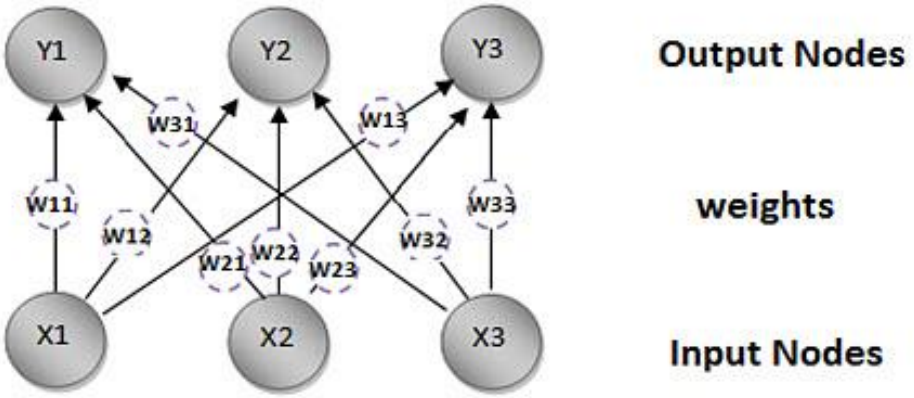


Fig 4.11: Simple Perceptron Architecture

Generally data is presented at the input layer, the network then processes the input by multiplying it by the corresponding weights present in each connection path. The inputs which are multiplied by the corresponding weights are then added together. Now if the sum goes above a preset threshold value the perceptron fires else it doesn't fire. This is done with the help on an *activation function* which is one of the key components of the perceptron. The basic model of a neuron used in Perceptrons and MLPs is the McCulloch-Pitts model, which dates from the late 1940s. This activation function used there is a simple threshold function as shown by equation 6.7.

$$f(x) = \begin{cases} 1 & x>0 \\ 0 & Otherwise \end{cases} \quad (6.7)$$

The main feature of the perceptron is its ability to get trained. The process of finding the correct values for the weights is called the learning rule, and the process involves initializing the weight matrix to a set of random numbers between -1 and +1. Then as the network learns, these values are changed until it has been decided that the network has solved the problem. Finding the correct values for the weights is effected using a learning paradigm called supervised learning. *Supervised learning is sometimes referred to as training*. Valid data is required to train the network; this constitutes input data for which the correct output is known. Starting with random weights, an input pattern is presented to the network; it makes an initial guess as to what the correct output should be. During the training phase, the difference `between the guess made by the network and the required value for the output is assessed, which is the error. Now weights are slightly adjusted by adding a percentage the error with the previous weights. After repeating this cycle a number of times, the perceptron will readjust its weights such that it correctly classifies any input given to it*. It is a fact that single perceptron can only model functions whose graphical models are linearly separable*. So the perception could easily imitate the logic of an AND gate or an OR gate. But it was a problem when it comes to training the perceptron for XOR logic which was non-linear in nature. But the Perceptrons enable a pattern to be broken up into simpler parts that can each be modeled by a separate perceptron in a network. So, even though perceptrons are limited, they can be combined into one powerful network that can model a wide variety of patterns, such as XOR and many complex Boolean expressions of more than one variable

Learning Algorithm

Backpropagation (back-error propagation) is the most widely used supervised learning algorithm in neural computing. A backpropagation network includes one or more hidden layers. This type of network is considered feedforward because there are no interconnections between the output of a processing element and the input of a node in the same layer or in a preceding layer. Externally provided correct patterns are compared with the neural network's output

during (supervised) training, and feedback is used to adjust the weights until the network has categorized all the training patterns as correctly as possible (the error tolerance is set in advance). Starting with the output layer, errors between the actual and desired outputs are used to correct the weights for the connections to the previous layer. For any output neuron j, $error = Z_j - Y_j$, where Z_j and Y_j are the desired and actual outputs, respectively. Using the sigmoid function $f = [1 - e^{-x}]^{-1}$, where x is proportional to the sum of the weighted inputs to the neuron, is an effective way to compute the output of a neuron in practice. This sigmoid function serves to keep the error correction well bounded within range [0, 1]. The weights of each input to the j_{th} neuron are then changed in proportion to this calculated error. A more complicated expression can be derived to work backward in a similar way from the output neurons through the hidden layers to calculate the corrections to the associated weights of the inner neurons. This complicated method is an iterative approach to solving a nonlinear optimization problem that is very similar in meaning to the one characterizing multiple-linear regression.

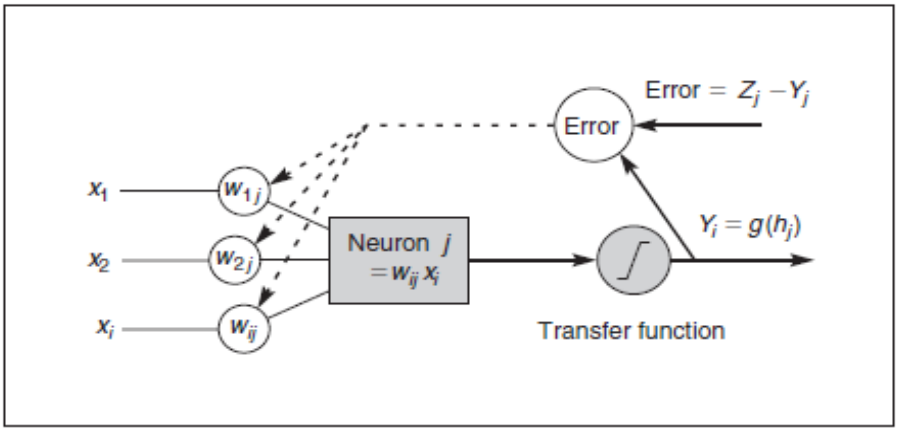


Fig 4.12: Pictorial representation – Learning algorithm

The learning algorithm includes the following procedures:

1. First of all initialize all the weights with random values and set other parameters.
2. Now read the entire input vector and the desired output from the training data set.
3. Compute the actual output via the calculations, working forward through the layers.
4. For each node n in the output layer, compute the error.

5. Change the weights by working backward from the output layer through the hidden layers.
6. Repeat the steps from step 2 after providing the next pattern in the training set as the new input until the error becomes 0 or less than a tolerance value.

Thus perceptron algorithm mainly aims at minimizing the error function. But it is always difficult to predict the time taken for the error function value to converge since the method of adding a portion of error term to the weights is an inefficient method. This is in the sense that the weights assume higher values as the number of iterations required for the error function to converge increases. In order to control the convergence rate, and reduce the size of the steps being taken, a parameter called a *learning rate (α)* is used. This parameter is set to a value that is less than unity, and means that the weights are updated in smaller steps. Sometimes, depending on the initial random weights and network parameters, the network does not converge to a satisfactory performance level. When this is the case, new random weights must be generated, and the network parameters, or even its structure, may have to be modified before another attempt is made. Current research is aimed at developing algorithms and using parallel computers to improve this process. For example, genetic algorithms can be used to guide the selection of the network structure [11].

Example of Perceptron Learning Algorithm

Lets for understanding purpose find out how the logic of OR gate is emulated by a Perceptron by using perceptron learning algorithms. The Truth Table of OR gate is given table 4.3.

Case	X1	X2	Z
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

Table 4.3: XOR truth table

As mentioned earlier, the main requirement for a supervised learning is availability of genuine data. And here for the OR gate, the very familiar truth table of OR gate provides the input data. Initially all the weights in middle layer of the neuron is set to a random value between 0 and 1 (W1=0.1 and W2=0.3). Each time Y_j is obtained, it is compared with the expected output and the difference between them is considered as the error factor (delta).

$$delta = Z_j - Y_j \tag{6.8}$$

Now depending on this delta factor the weights are adjusted correspondingly. The new weights are found out the by the equation 6.9.

$$W_i(final) = W_i(initial) + alpha * delta * X_i \tag{6.9}$$

The input patterns are continuously applied and the weights get adjusted in successive iteration. This step is repeated until the weights converge to a uniform set of values that allow the neuron to classify each of the four inputs correctly. Here alpha (called learning rate) is a parameter that controls how fast the learning takes place. A high value for the learning rate can lead to too much correction in the weight values, resulting in going back and forth among possible weights values and never reaching the optimal, which may lie somewhere in between the endpoints. Too low a learning rate may slow down the learning process. In practice, a neural network analyst may try using many different choices of learning rates to achieve optimal learning. Now the learning process is over and the system is ready to accept any new input, for which output will be correctly predicted [14].

4.6.3 Recurrent Network

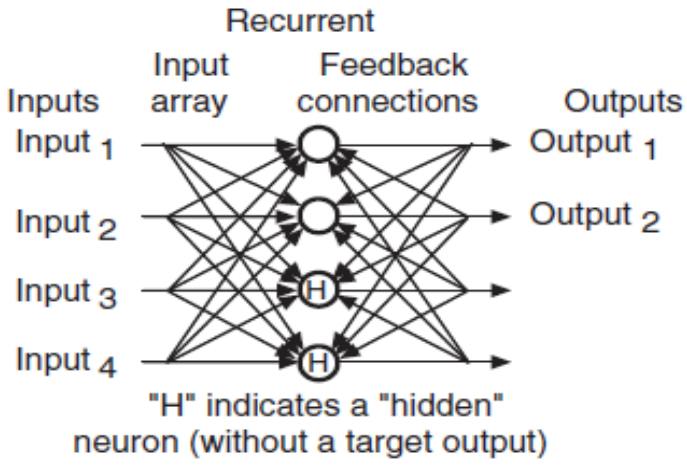


Fig 4.13: Recurrent Network

In recurrent network all the input layer nodes are connected to all the middle layer nodes(neurons) and also all the output layer nodes are connected to all the middle layer nodes. Thus recurrent network have a feedback element that enables signals from one layer to be fed back to the previous layer. *A simple recurrent network* consists of an input layer, a middle layer and an output layer. A set of additional context units are added to the input layer that receive input from the hidden layer neurons. The feedback path from the hidden layer to the context unit has a fixed weight of unity. A *fully recurrent network* is one where every neuron receives input from all other neurons in the system and due its complexity such network cannot be easily arranged into layers. Here a small subset of neurons receives external inputs and another small subset produces system output. A recurrent network is known as symmetrical network if.

$$W_{ij} = W_{ji} \quad \text{For all i, j} \tag{6.10}$$

Where W_{ij} the weight on the path between the input and the middle is layer and W_{ji} is weight in the path between corresponding output layer and middle layer. Human brain is a recurrent neural network. It can learn many behaviors, sequence processing tasks, algorithms, programs that are not learned by traditional machine learning methods. This explains the rapidly growing interest in *artificial* RNNs for technical applications general computers which can learn algorithms to map input sequences to output sequences, with or without a teacher. They are computationally more powerful and biologically more plausible than other adaptive approaches

such as Hidden Markov Models (no continuous internal states), feedforward networks and Support Vector Machines (no internal states at all). The main applications areas of recurrent network includes adaptive robotics and control, handwriting recognition, speech recognition, keyword spotting, music composition, attentive vision, protein analysis, stock market prediction, and many other sequence problems. Early RNNs of the 1990s could not learn to look far back into the past. Their problems were first rigorously analyzed on Schmidhuber's RNN long time lag project by his former PhD student Hochreiter (1991). A feedback network called "Long Short-Term Memory" (LSTM, Neural Comp., 1997) overcomes the fundamental problems of traditional RNNs, and efficiently learns to solve many previously unlearnable tasks involving:

1. Recognition of temporally extended patterns in noisy input sequences
2. Recognition of the temporal order of widely separated events in noisy input streams
3. Extraction of information conveyed by the temporal distance between events.
4. Stable generation of precisely timed rhythms, smooth and non-smooth periodic trajectories.
5. Robust storage of high-precision real numbers across extended time intervals

4.6.4 Associative Learning Architecture

The function of an associative memory is to recognize previously learned input vectors, even in the case where some noise has been added. With respect to neural network, learning means classifying an input vector ‘x’ to an output ‘y’. In associative learning we classify a set of input vectors V(x) such that the distance of each vector, which is an element of set V(x), to the input vector ‘x’ is less than a constant positive value ‘e’ [13].

For example, we see a set of photographs of certain people when we were young and it gets embedded in our memory. Then after say 5 years, we see the photographs of the same people. We observe that their physical attributes have changed. According to the definition of the associative learning problem, if the physical changes between the person’s image which is embedded in our memory and image we see in photographs after five years are below a certain error threshold then we can correctly associate or recognize the person.

Associative memories can be implemented using networks with or without feedback, but the latter produce better results. There are three kinds of associative networks [5].

1) Heteroassociative networks, map m input vectors $x_1, x_2, \ldots, x_m$ in 'n' dimensional space to m output vectors $y_1, y_2, \ldots, y_m$ in k-dimensional space, so that $x^i \mapsto y^i$ i. If $\left\| \tilde{x} - x^i \right\|^2 < \varepsilon$ then $\tilde{x} \mapsto y^i$. This should be achieved by the learning algorithm, but becomes very hard when the number m of vectors to be learned is too high.

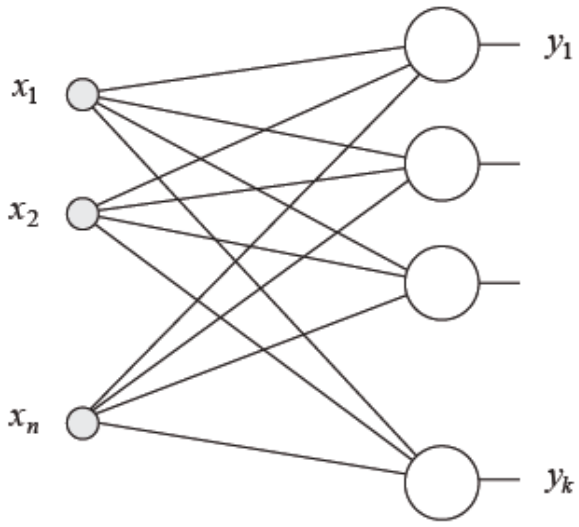


Fig 4.14: Heteroassociative network without feedback

2) Autoassociative networks are a special subset of the heteroassociative networks, in which each vector is associated with itself, i.e., $y_i = x_i$ for i = 1, . . .,m. The function of such networks is to correct noisy input vectors.

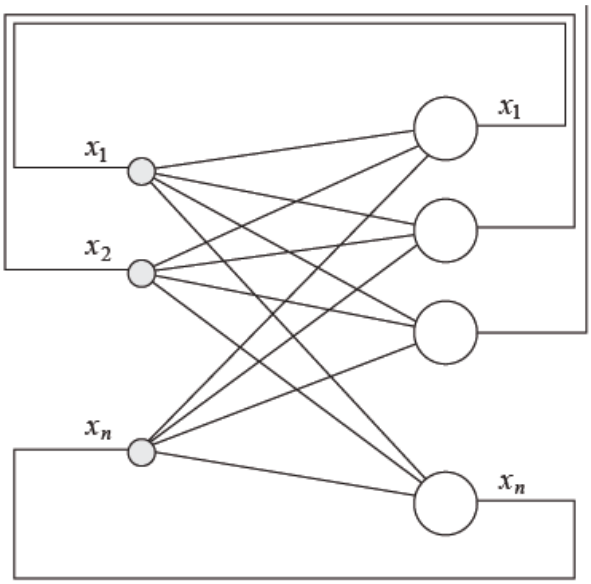


Fig 4.15: Autoassociative network with feedback

3) Pattern recognition networks are also a special type of heteroassociative networks. Each vector x_i is associated with the scalar i. The goal of such a network is to identify the 'name' of the input pattern.

4.7 Artificial Neural Network Types

4.7.1 Multilayer Perceptrons (MLP)

A multilayer perceptron has an input layer of source nodes and an output layer of neurons (i.e., computation nodes); these two layers connect the network to the outside world. In addition to these two layers, the multilayer perceptron usually has one or more layers of hidden neurons (hidden layer), which are so called because these neurons are not directly accessible. The hidden neurons extract important features contained in the input data [11].

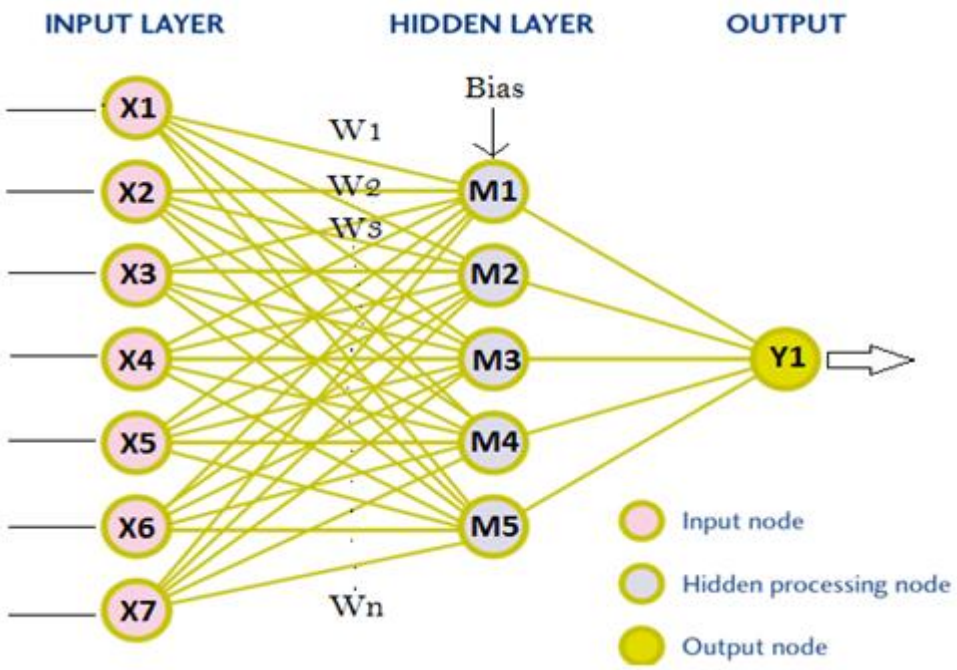


Fig4.16: Multilayer perceptrons

It is seen that the perceptron could easily solve problems that are linearly separable. But the principle weakness of the perceptron units with a threshold activation function could only solve problems that were linearly separable. But all the real life problems will not give data set which is linearly separable e.g. Assume a case where we need to emulate logic of an exclusive-or (XOR) gate .Now here we require a non linear classifier. The figure 4.17 shows the plot of all data values for XOR logic. As you can see it is not possible to obtain a linear classifier in this case. [By a linear classifier we mean a line which could separate the two classes]

Case	X1	X2	Z
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

Table 4.4: XOR Truth Table

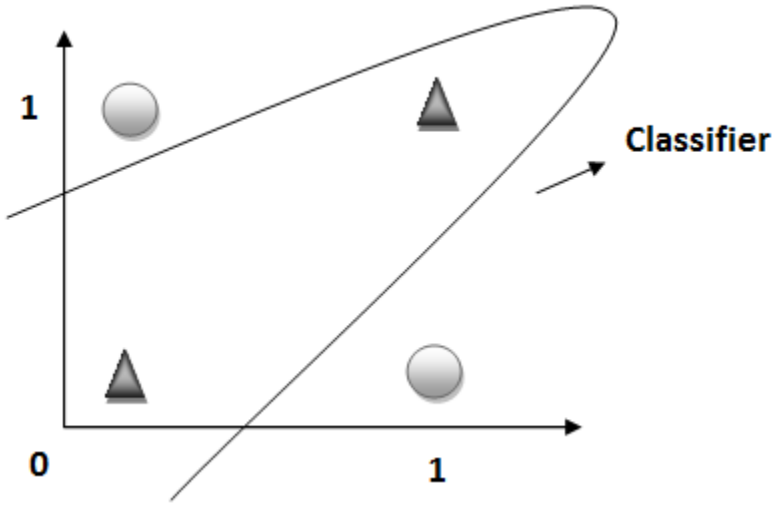


Fig 4.17: XOR Data distribution & nonlinear classifier

A better solution to the problem of learning weights is to use standard optimization techniques. In this case we identify an error function which is expressed in terms of the neural network output. The goal of the network then becomes to find the values for the weights such that the error function is at its minimum value. Thus gradient descent techniques can then be used to determine the impact of the weights on the value of the error function. We need to have an error function that is differentiable, which means it should be continuous. The threshold function is not continuous, and so is unsuitable. A function that works in a similar way to the threshold function, but that is differentiable is the Logistic Sigmoid Function, given by the following equation 6.11.

$$Y_T = \frac{1}{1+e^{-Y}} \qquad (6.11)$$

Where, Y is the output of Summation Function, Y^T is the normalized output. Here λ refers to a bias term which determines the steepness of the slope of the function. This function behaves in a very similar way to the threshold function, with x values above zero tending to unity, and values below zero tending to zero. This function is continuous, and hence it is differentiable. Because the function is differentiable, it is possible to develop a means of adjusting the weights in a perceptron over as many layers as may be necessary.

MLP learning Algorithm

The basic MLP learning algorithm is outlined below. This is what you should attempt to implement.

1. Initialize the weights in the network by random numbers between -1 and +1.
2. Provide the first training data to the input layer, and obtain the output.
3. Compare the response the perceptron gives with the required output.
4. Make adjustment to the output layer weights by the below said method

$$W_{ho} = W_{ho} + \left(\alpha * \delta_0 * O_h\right) \tag{6.12}$$

Where W_{ho} is the weight connecting hidden unit, h with output unit 0, α is the learning rate, O_h is the output at hidden unit h. δ_0 is given by the following.

$$\delta_0 = O_0\left(1 - O_0\right)\left(t_0 - O_0\right) \tag{6.13}$$

Where O_0 is the output at node 0 of the output layer, and t_0 is the target output for that node.

5. Make adjustment to the input weights using the following method

$$W_{ih} = W_{ih} + \left(\alpha * \delta_h * O_i\right) \tag{6.14}$$

where W_{ih} is the weight connecting node i of the input layer with node h of the hidden layer, O_i is the input at node i of the input layer, α is the learning rate. δ_h is calculated as follows.

$$\delta_h = O_h\left(1 - O_h\right)\sum_o\left(\delta_o W_{ho}\right) \tag{6.15}$$

6. Calculate the error, by taking the average difference between the target and the output vector. Here p is the number of units in the output layer.

$$E = \frac{\sqrt{\sum_{n=1}^{p}\left(t_o - O_o\right)^2}}{p} \tag{6.16}$$

7. Repeat from 2 for each pattern in the training set to complete one epoch.
8. Shuffle the training set randomly. This is important so as to prevent the network being influenced by the order of the data.
9. Repeat from step 2 for a set number of epochs, or until the error ceases to change.

4.7.2 Radial-Basis Function Networks

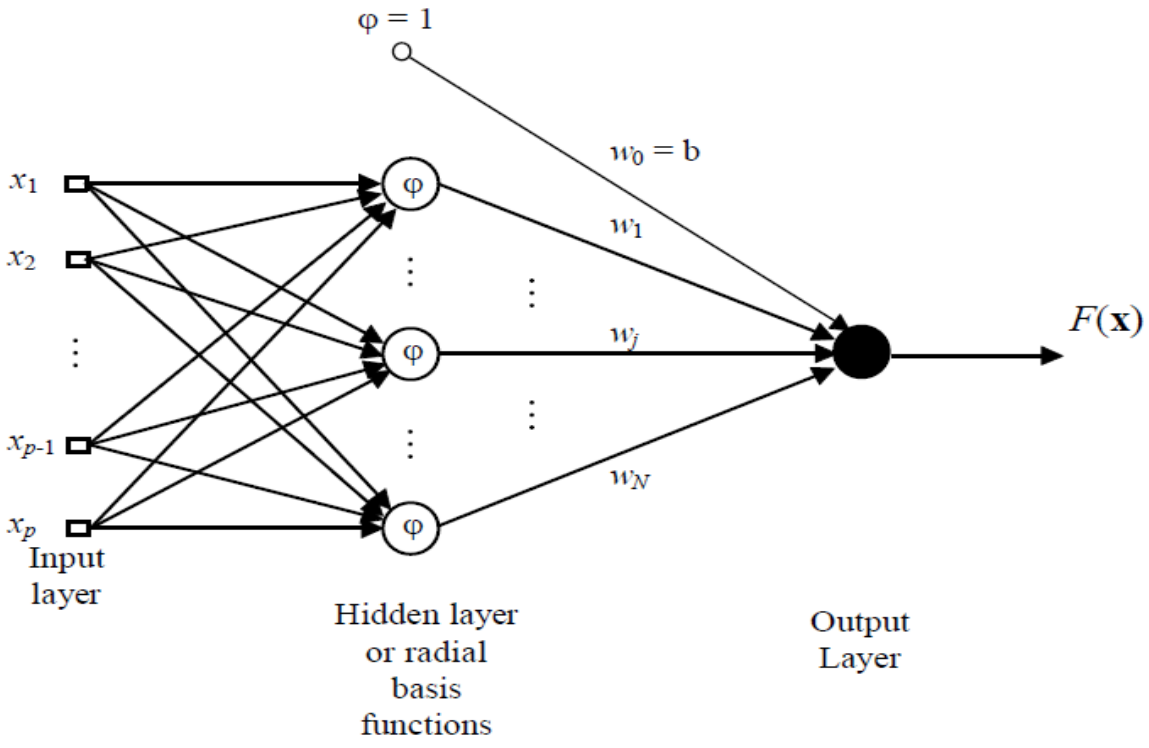


Fig 4.18: RBF Network

Radial basis function is one variation of artificial neural network which was introduced in the 1980's. As shown in Figure 4.18, the RBF network has three layered architecture. It has an input layer as usual, a single middle layer and an output layer. All the input nodes are connected to each node in the middle layer. Each node in the middle layer implements a radial activated function. The output is obtained by multiplying the middle layer node outputs with corresponding weights and then adding them together[13].

$$\Psi_k(x) = \sum_{j=1}^{L} w_{jk}\varphi_j(x) \tag{6.17}$$

Where $j = 1.....L$, L is number of hidden layers, X is input feature vector, w_{jk} is the output weights and $\varphi_j(x)$ is the response of middle layer nodes. The input to the RBF network is nonlinear. Unlike the multilayer perceptrons the output for RBF network is always linear. According to Cover's theorem, if the mapping from the input space to the feature space is nonlinear and the dimensionality of the feature space is high compared to that of the input (data) space, then there is a high likelihood that a non-separable pattern classification task in the input space is transformed into a linearly separable one in the feature space. Thus the nonlinear approximation property allows the RBF network to emulate complex functionalities which in the case of multilayer perceptron would have required multiple middle layers. As far as RBF networks are concerned the training process involves finding and fixing optimal RBF weights. The activation function of the hidden layer in an RBF network computes the Euclidean distance between the input signal vector and parameter vector of the network, whereas the activation function of a multilayer perceptron computes the inner product between the input signal vector and the pertinent synaptic weight vector. As per Simon Haykin "*RBF networks are local approximators, whereas multilayer perceptrons are global approximators*". RBF Networks are being used in wide variety of applications such as chaotic time series modeling, speech reorganization, control systems, image processing, data fusion etc.

Activation Function for RBF network

The selection of activation function depends on the application where the RBF Network is used.

a) For pattern classification applications the output is restricted to [0, 1] by sigmoid function (equation. 6.18).

$$f(x) = \frac{1}{1+e^{-x}} \tag{6.18}$$

Where x is the weighed sum of the response of nodes in the middle layer.

b) Another activation function used in RBF network is the Gaussian activation function is shown in equation 6.19.

$$f(x) = e^{\left[-(X-\mu_j^T)\sum_j^{-1}(X-\mu_j)\right]} \qquad (6.19)$$

Where $j = 1.....L$ L is number of hidden layers, X is input feature vector, μ_j and Σ_j is the mean and covariance of j[th] Gaussian Function.

NOTE

For time series modeling "thin plate spline" is used as the activation function.

4.7.3 Hopfield Network

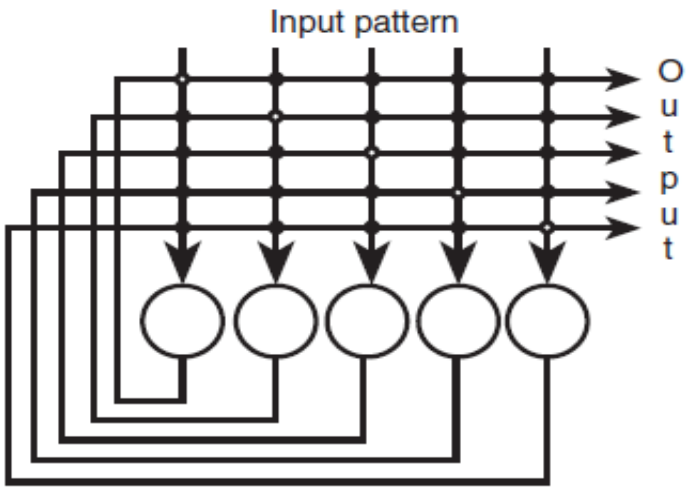


Fig 4.19: HopField Network

One of the neural network models of interest is the Hopfield network (Hopfield, 1982). John Hopfield showed in a series of papers in the 1980s how highly interconnected networks or nonlinear neurons can be extremely effective in computing. These networks provided a rapid computed solution for problems stated in terms of desired optima, often subject to constraints. A general Hopfield network is a single large layer of neurons with total interconnectivity—that

is, each neuron is connected to every other neuron. In addition, the output of each neuron may depend on its previous values. One use of Hopfield networks has been in solving constrained optimization problems, such as the classic traveling salesman problem (TSP). In this type of application, each neuron represents the desirability of a city 'n' being visited in position 'm' of a TSP tour. Interconnection weights are specified, representing the constraints of feasible solution to the TSP (e.g., forcing a city to appear in a tour only once). An energy function is specified, which represents the objective of the model solution process (e.g., minimize total distance in the TSP tour) and is used in determining when to stop the neural network evolution to a final state. The network starts with random neuron values and, using the stated interconnection weights, the neuron values are updated over time. Gradually, the neuron values stabilize, evolving into a final state (as driven by the global energy function) that represents a solution to the problem. At this point in the network evolution, the value of neuron (n,m) represents whether city 'n' should be in location 'm' of the TSP tour. While Hopfield and Tank (1985) and others claimed great success in solving the TSP, further research has shown those claims to be somewhat premature. Nonetheless, this novel approach to a classic problem offers promise for optimization problems, especially when technology allows for taking advantage of the inherent parallelism of neural networks. Hopfield networks are distinct from feedforward networks because the neurons are highly interconnected, weights between neurons tend to be fixed, and there is no training per se. The complexity and challenge in using a Hopfield network for optimization problems is in the correct specification of the interconnection weights and the identification of the proper global energy function to drive the network evolution process [13].

4.7.4 Kohonen Neural Network

Self organizing feature maps (SOFM or Kohonen) networks are used quite differently to the other networks. Whereas all the other networks are designed for supervised learning tasks, SOFM networks are designed primarily for unsupervised learning (Patterson 1996). At first glance this may seem strange. Without outputs what can the network learn? The answer is that

the SOFM network attempts to learn the structure of the data. One possible use in therefore in exploratory data analysis. A second possible use is in novelty detection. SOFM networks can learn to recognize clusters in the training data and respond to it. If new data, unlike previous case, is encountered, the network fails to recognize it and this indicates novelty. SOFM networks has only two layers – the input layer and output layer of radial units(topological map).

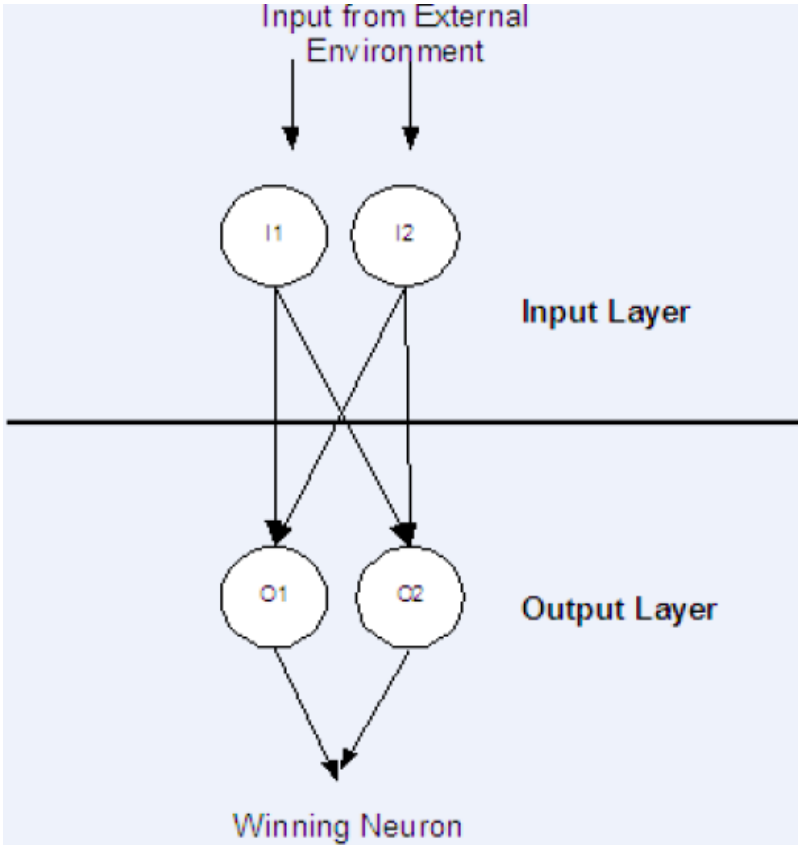


Fig 4.20: A Kohonen Neural Network Application

4.8 ANN Design Flow

Although the development process of ANN is similar to the structured design methodologies of traditional computer-based information systems, some phases are unique or have some unique aspects. In the process described here, we assume that the preliminary steps of system development, such as determining information requirements, conducting a feasibility

analysis, and gaining a champion in top management for the project, have been completed successfully. Such steps are generic to any information system.

As shown in Figure 4.21, the development process for an ANN application includes nine steps. In step 1, the data to be used for training and testing the network are collected and preprocesed. Important considerations are that the particular problem is amenable to neural network solution and that adequate data exist and can be obtained. In step 2, training data must be identified, and a plan must be made for testing the performance of the network.

In steps 3 and 4, the network architecture and a learning method are selected. The availability of a particular development tool or the capabilities of the development personnel may determine the type of neural network to be constructed. Also, certain problem types have demonstrated high success rates with certain configurations (e.g., multilayer feedforward neural networks for bankruptcy prediction, as described in the next section). Important considerations are the exact number of neurons and the number of layers. Some packages use genetic algorithms to select the network design.

There are parameters for tuning the network to the desired learning-performance level. Part of the process in step 5 is the initialization of the network weights and parameters, followed by the modification of the parameters as training-performance feedback is received. Often, the initial values are important in determining the efficiency and length of training. Some methods change the parameters during training to enhance performance.

Step 6 transforms the application data into the type and format required by the neural network. This may require writing software to preprocess the data or performing these operations directly in an ANN package. Data storage, manipulation techniques and processes must be designed for conveniently and efficiently retraining the neural network, when needed. The application data representation and ordering often influence the efficiency and possibly the accuracy of the results.

In steps 7 and 8, training and testing are conducted iteratively by presenting input and desired or known output data to the network. The network computes the outputs and adjusts

the weights until the computed outputs are within an acceptable tolerance of the known outputs for the input cases. The desired outputs and their relationships to input data are derived from historical data (i.e., a portion of the data collected in step 1).

In step 9, a stable set of weights is obtained. Now the network can reproduce the desired outputs, given inputs such as those in the training set. The network is ready for use as a standalone system or as part of another software system where new input data will be presented to it and its output will be a recommended decision. In the following sections, we examine these steps in more detail.

4.8.1 Data Collection and Pre-Processing

ANN is actually a system which mocks the functionality and operation of the system under study. Also as mentioned previously a valid dataset (combination of actual inputs and obtained output for the system under study) is the primary requirement of developing the ANN application. The first two steps in the ANN development process involve collecting data, pre-processing it and then separating them into a training set and a testing set. The training cases are used to adjust the weights in the neural network architecture, and the test cases are used for network validation. The data used for training and testing must include all the attributes that are useful for solving the problem. The system can only learn as much as the data can tell. Therefore, collection and preparation of data is the most critical step in building a good system.

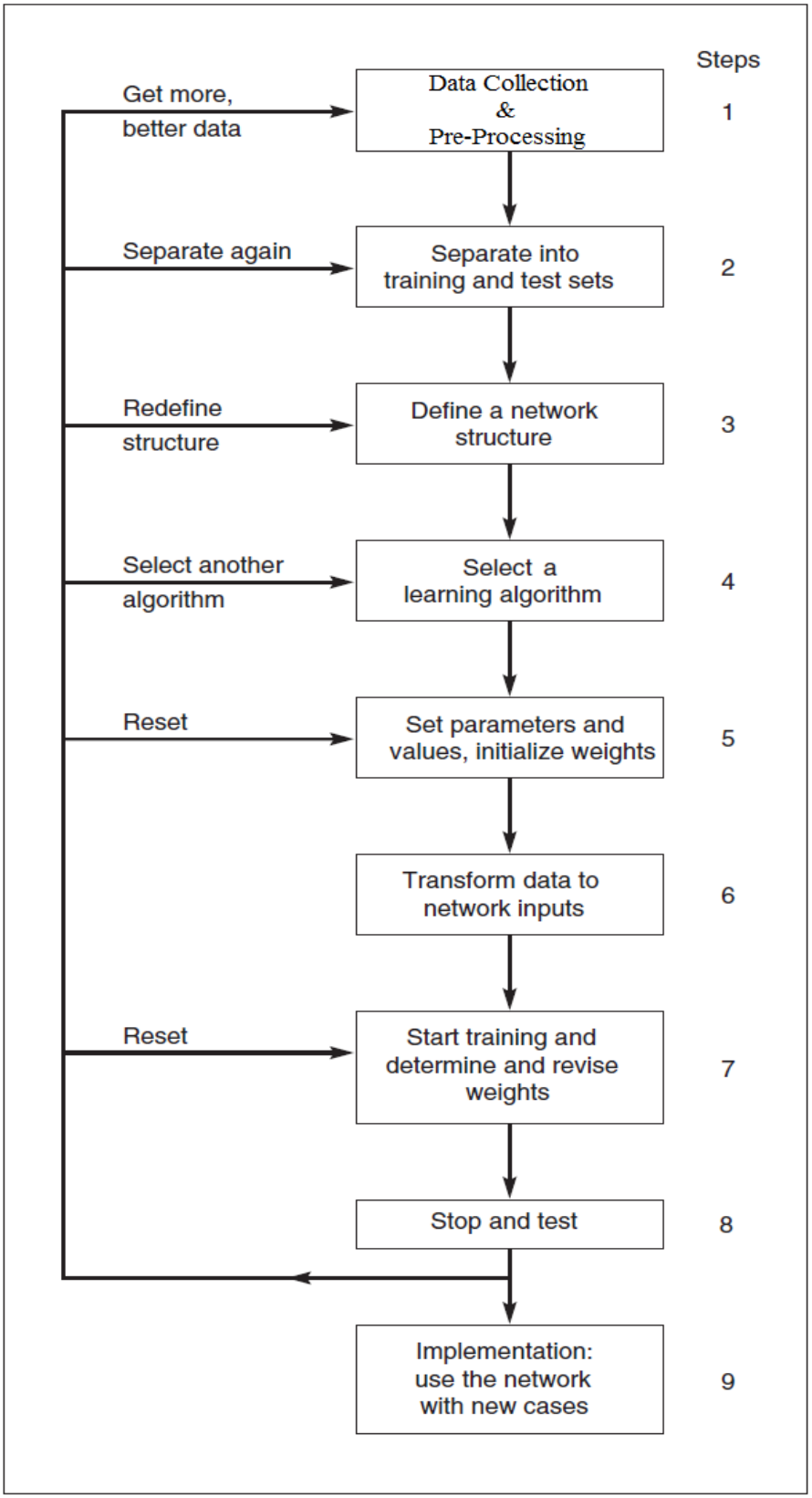


Fig 4.21: Basic design flow for neural network

In general, the more data used the better. Larger data sets increase processing time during training but improve the accuracy of the training and often lead to faster convergence to a good set of weights. For example, say a bank wants to build a neural network–based system in order to use clients' financial data to determine whether they may go bankrupt. The bank needs to first identify what financial data may be used as inputs and how to obtain them. Five attributes may be useful inputs: (1) working capital/total assets, (2) retained earnings/total assets, (3) earnings before interest and taxes total assets, (4) market value of equity/total debt, and (5) sales/total sales. The output is a binary variable: bankruptcy or not [14].

NOTE

The data requirement for neural network is dependent on the type of application for which the neural network is trained. If the application demands greater accuracy, only relevant data can be selected and so on.

The data collection mainly involves three main processes. They include

1) **Identifying the data requirement**- Since the performance of neural network depends on the data based on which it is trained, the first thing to do when planning data collection is to decide what data we will need to solve the problem. In general, it will be necessary to obtain the assistance of some experts in the field for which the application is being developed. All the definitely relevant, possibility relevant, collateral data need to be collected. Usually both relevant and possibly relevant data should be considered as inputs to the application.
2) **Identifying data sources** – The accuracy of data is one thing that cannot be compromised in case of neural network. The data has to be accurate at the same time should be available in bulk. Now we need to decide from where the data need to be obtained. This will allow us to make realistic estimates of the difficulty and expense of obtaining it. If the application demands real time data, these estimates should include an allowance for converting analogue data to digital form. In some cases, it may be desirable to obtain data from a simulation of the real situation. This could be the case if

the application is intended to monitor conditions which have health, safety or significant cost implications. Care must be taken to ensure that the simulation is accurate and representative of the real case.

3) **Determining the data quantity** - It is important to make a reasonable estimation of how much data we will need to develop the neural network properly. If too little data is collected, it may not reflect the full range of properties that the network should be learning, and this will limit its performance with unseen data. On the other hand, it is possible to introduce unnecessary expense by collecting too much data. In general, the quantity of data required is governed by the number of training cases that will be needed to ensure the network performs adequately. The intrinsic dimensionality of the data and the required resolution are the main factors determining the number of training cases and, therefore, the quantity of data required. It is vital to assess correctly the quality of the data that will be presented to the neural network. Often, the data will be less than perfect, and if the network is to perform correctly then it needs to be trained with a greater quantity of data than would be the case if high quality data were available.

In order to facilitate the training procedure and to increase the performance of neural networks a five-step algorithm of pre-processing of input/output data is used. The stages of the algorithm include the estimation and replacement of outliers, testing for stationarity, the evaluation of seasonal components, testing for normality and scaling to the range of reasonable values of the activation function.

4.8.1.1 Data Pre-Processing

The performance of an ANN application is highly dependent on inputted data. Thus the raw input data always need to be preprocessed; or otherwise the neural network will not produce accurate forecast. Transformation and normalization are the two major preprocessing methods. A more complex pre-processing operation may involve the reduction of dimensionality of the input data [3]. Transformation refers to manipulating raw data inputs to

create a single input or it can also be transformation of categotical variables or other non numerical variables to numerical variables. Some simple transformation methods include computing difference between two input data or taking the ratio of input. This reduces the number of inputs to the neural network and hence helps it learn more easily. In case of time series analysis the output of moving average algorithm can be given as the input data to the neural network; this helps in giving useful information which is free from noise, at the same time a reasonable level is also maintained. While normalization is the transformation performed on each data input to distribute the data evenly and scale it into an acceptable range for the network. There are many normalization method available. One of them is the ,simple linear scaling, where the data is scaled into a particular range comfortable with the neural network. Usually the input data's are scaled to a range [-1, 1] or [0, 1]. Many commercially available generic neural network development software's such as NeuralWorks, Brainmaker and DynaMind automatically scales the input. In another normalization method, it utilizes a statistical measure of central tendency and variance to help remove outliers, and spread out the distribution of the data, which tends to increase uniformity. This is a relatively simple method of normalization, in which the mean and standard deviation for the input data associated with each input are determined. There are other methods for data normalization. Some methods are more appropriate than others, depending on the nature and characteristics of the data to be normalized. When the network is run on a new test fact, the output produced must be de-normalized. If the normalization is entirely reversible with little or no loss in accuracy, then there is no problem. However, if the original normalization involved clipping outlier values, then output values equal to the clipping boundaries should be suspect concerning their actual value. For example, assume that during training ail output values greater than 50 were clipped. Then, during testing, if the net produces an output of 50, this indicates only that the net's output is 50 or greater. If that information is acceptable for the application, then the normalization method would be sufficiently reversible [14].

4.8.2 Separating into Training and Testing Data

Training and testing is the backbone of neural network. As mentioned apriori, the amount of training data and the quality of training data always affect the performance of the neural network. From the collected data we now need to allocate data for training as well as testing purpose. The data based on which the neural network is trained is not usually given as test data. For a moderately sized data set, typically 80 percent of the data are randomly selected for training and 20 percent are selected for testing; for small data sets, typically all the data are used for training and testing; and for large data sets, a sufficiently large sample is taken and treated like a moderately sized data set.

4.8.3 Selection of Network Structure

The structure of ANN is not always the same for all applications. The selection of topologies to be used, number of input nodes, number of output nodes, number of hidden layers and .number of hidden nodes has to be done suitabilyThe multilayer feedforward topology is often used in business applications, although other network models are beginning to find some business use as well.The design of input nodes must be based on the attributes of the data set. In the example of predicting bankruptcy, for example, the bank might choose a three-layer structure that includes one input layer, one output layer, and one hidden layer. The input layer contains five nodes, each of which is a variable, and the output layer contains a node with 0 for bankrupt and 1 for safe. The number of middle layers can be chosen arbitrarily, although the most common is one layer. It is a fact that, including more middle layers will increase the computational complexity of the network. So there are various factor which are taken into account while selecting the number of middle layers e.g. complexity of the problem, memory constraint etc. But for most of the nonlinear problem such as games require only one middle layer. Determining the number of hidden nodes is tricky. Unlike the output layer node selection the number of hidden units is not bound to any explicit characteristics. They can have a great effect on the performance of the network. A few heuristics have been proposed, but none of them is unquestionably the best. A typical approach is to choose the average number of input and output nodes. In the previous case, the hidden node may be set to (5 - 1)/2 -7. Figure 4.22

shows a possible structure for the bankruptcy-prediction problem. For complex problems, the number of hidden layers required can grow fast than exponentially. Radial-Basis Function Networks, Recurrent network, Hopfield network and Association network are the other neural network architectures available [14].

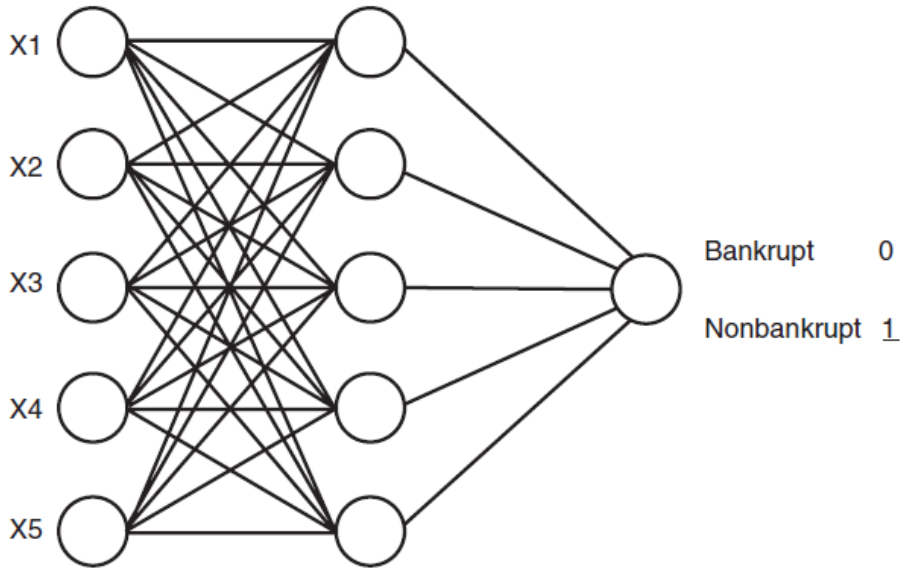


Fig 4.22: ANN architecture for predicting bankruptcy

4.8.4 Learning Algorithm Selection

After the network structure is chosen, we need to find a learning algorithm to identify a set of connection weights that best cover the training data and have the best predictive accuracy. For the bankruptcy-prediction problem, a typical approach is to use the backpropagation algorithm. Because many commercial packages are available on the market, there is no need to implement the learning algorithm by ourselves. Instead, we can choose a suitable commercial package to analyze the data. There are different types of neural network software packages such as Stuttgart Neural Network Simulator (SNNS), Emergent, JavaNNS and Neural Lab which are available currently.

4.8.5 ANN Training

Training of ANN is an iterative process that starts from a random set of weights and gradually enhances the fitness of the network model and the known data set. The iteration continues until the error sum is converged to a preset acceptable level. In the backpropagation algorithm, two parameters, learning rate and momentum, can be adjusted to control the speed of reaching a solution. These determine the ratio of the difference between the calculated value and the actual value of the training cases. Some software packages may have their own parameters in their learning heuristics to speed up the learning process. It is important to read carefully when using this type of software. Some data conversion may be necessary in the training process. This includes (1) changing the data format to meet the requirements of the software, (2) normalizing the data scale to make the data more comparable, and (3) removing problematic data. When the training data set is ready, it is loaded into the package, and the learning procedure is executed. Depending on the number of nodes and the size of the training

data set, reaching a solution may take from a few thousand to millions of iterations.

4.8.6 Testing of ANN

Recall that in step 2 of the development process shown in Figure 4.21, the available data are divided into training and testing data sets. When the training has been completed, it is necessary to test the network. Testing (step 8) examines the performance of the derived network model by measuring its ability to classify the testing data correctly. Black-box testing (i.e., comparing test results to historical results) is the primary approach for verifying that inputs produce the appropriate outputs. Error terms can be used to compare results against known benchmark methods. The network is generally not expected to perform perfectly (zero error is difficult, if not impossible, to attain), and only a certain level of accuracy is really required. For example, if 1 means nonbankrupt and 0 means bankrupt, then any output between 0.1 and 1 might indicate a certain likelihood of nonbankrupty. The neural network application is usually an alternative to another method that can be used as a benchmark against which to compare accuracy. For example, a statistical technique such as multiple regressions or

another quantitative method may be known to classify inputs correctly 50 percent of the time. The neural network implementation often improves on this. For example, Liang (1992) reported that ANN performance was superior to the performance of multiple discriminant analysis and rule induction. Ainscough and Aronson (1999) investigated the application of neural network models in predicting retail sales, given a set of several inputs (e.g., regular price, various promotions).They compared their results to those of multiple regression and improved the adjusted R2 (correlation coefficient) from .5 to .7. If the neural network is replacing manual operations, performance levels and speed of human processing can be the standard for deciding whether the testing phase is successful. The test plan should include routine cases as well as potentially problematic situations. If the testing reveals large deviations, the training set must be reexamined and the training process may have to be repeated (some "bad" data may have to be omitted from the input set). Note that we cannot equate neural network results exactly with those found using statistical methods. For example, in stepwise linear regression, input variables are sometimes determined to be insignificant, but because of the nature of neural computing, a neural network uses them to attain higher levels of accuracy. When they are omitted from a neural network model, its performance typically suffers [14].

4.8.7 Implementation of ANN

Implementation of an ANN (step 9) often requires interfaces with other computer based information systems and user training. Ongoing monitoring and feedback to the developers are recommended for system improvements and long-term success. It is also important to gain the confidence of users and management early in the deployment to ensure that the system is accepted and used properly.

NOTE

Knowledge of the domain is important in choosing preprocessing method.

4.9 Summary

Artificial Neural Network is one of the first machines learning methodology introduced. The inspiration for the development the ANN model was obtained from biological nervous systems, such as the brain works. ANNs are usually used for pattern recognition or data classification. ANN implementations are now utilized broadly in finance, marketing, manufacturing, operations, and information systems. The chapter gives an idea about the working of ANN, ANN architecture, ANN leaning process, the types of ANN, different activation functions and finally about the design steps in developing an ANN system.

Questions:

1. Explain the concept of Artificial Neural Network? How is it connected to Biological Neural Systems?
2. Explain the following terms: *neuron*, *axon*, and *synapse*?
3. How do weights function in an ANN?
4. What is the role of the summation function?
5. What is an activation function? How does it help in the working of ANN?
7. Explain in details the various steps involves in designing an ANN?

References

[1] Christos Stergiou and Dimitrios Siganos. "Neural Networks" , Imperial College of London, Computer Department; http://www.doc.ic.ac.uk/~nd/surprise_96/journal/vol4/cs11/report.html#Contents
[2] Ajith Abraham , Artificial Neural Network Oklahoma State University, Stillwater, OK, USA
[3] Bishop, C.M. (1995) Neural Networks for Pattern Recognition, Oxford University Press, Oxford, UK.
[4] Laurence Fausett, "Fundamentals of Neural Networks Architectures, Algorithms, and Applications", Prentice Hall Upper Saddle River, New Jersey 1994.
[5] R. Rojas: Neural Networks, Springer-Verlag, Berlin, 1996 -- Associative network
[6] Insight into Data Mining. Theory and Practice with CD ROM. *K.P. Soman*. Shyam Diwakar. V. Ajay. 510.78. S6 I6. 206291 2. 188
[7] Minsky, M. and Papert, S. (1969). *Perceptrons*. MIT Press, Cambridge.
[8] McCulloch, W. and Pitts, W. (1943). *A logical calculus of the ideas immanent in nervous activity.* Bulletin of Mathematical Biophysics
[9] Hahn, B., et al. Neuroanatomical dissociation between bottom-up and top-down processes of visuospatial selective attention. NeuroImage 2006; 32:842-853
[10] Elements of Artificial Neural Networks - Kishan Mehrotra, Chelkuri K. Mohan, and Sanjay Ranka, Penram International.

[11] Artificial Neural Network – Simon Haykin, Pearson Education, 2nd Ed.
[12] Rutkowska D.: Neuro-Fuzzy Architecture and Hybrid Learning, Physica-Verlag Press (2002)
[13]Neural Networks - A Systematic Introduction, a book by Raul Rojas Foreword by Jerome Feldman Springer-Verlag, Berlin, New-York, 1996.
[14] E. Turban, R. Sharda, J. Aronson and D. King. Business Intelligence: A Managerial Approach, 1st Edition, Prentice Hall, Upper Saddle River, NJ, 2008, Chapter 2- Data Warehousing – contributing author.